Astro Game Development Kits Overview

For AstroGDK 2.9

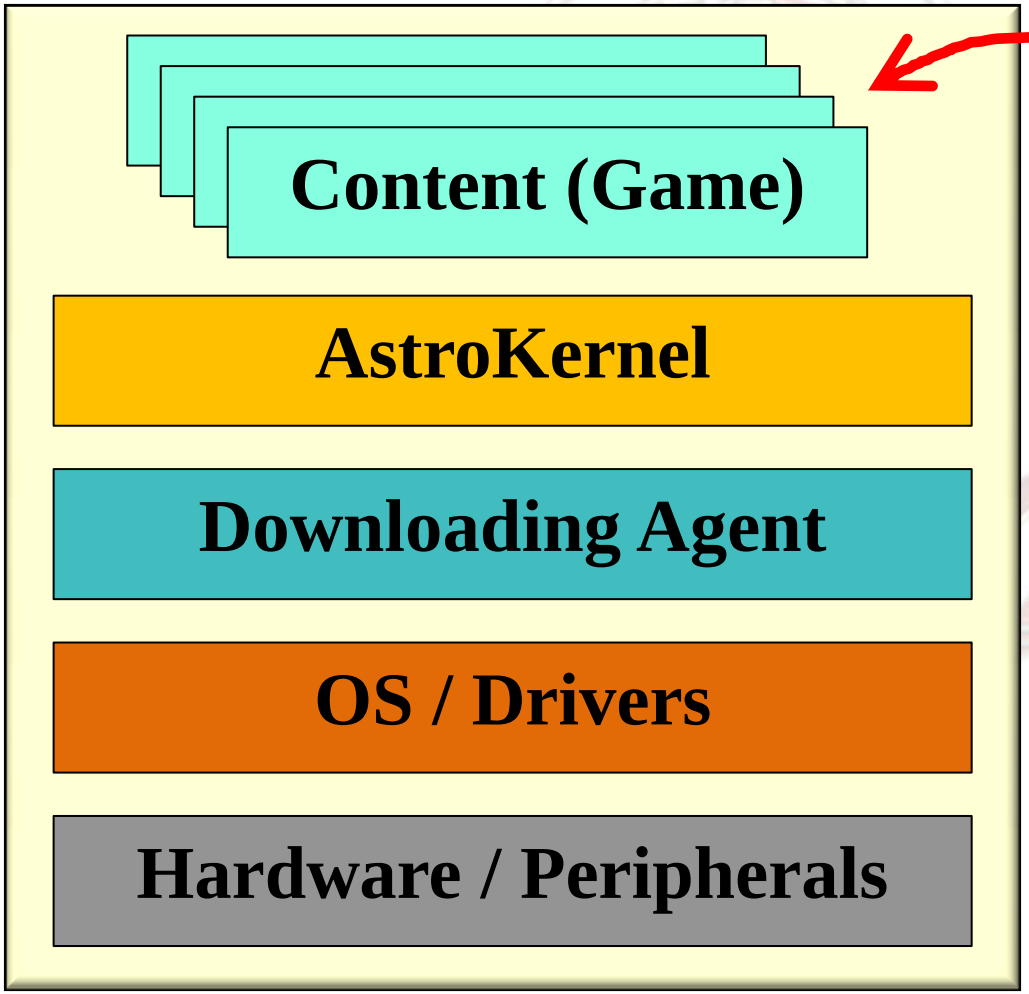
by Tseng, Ray, Chance

Preface

- This is an introduction of Astro Game Development Kits (AstroGDK) for software engineers relating to game programming.

- Agenda
  - VLT Software Architecture
  - AstroGDK – Introduction
  - AstroGDK – Setup
  - AstroGDK – Programming (AK2API and Sample Code)
  - AstroGDK – Run and Test
  - AstroGDK – Game Submission

VLT Software Architecture



You develop this.

VLT Software Architecture

- Downloading agent
  - The process is launched at the beginning handling tasks of
    - Downloading, installing and/or updating contents (games), and then
    - Checking software integrity, and eventually
    - Launching process “AstroKernel”

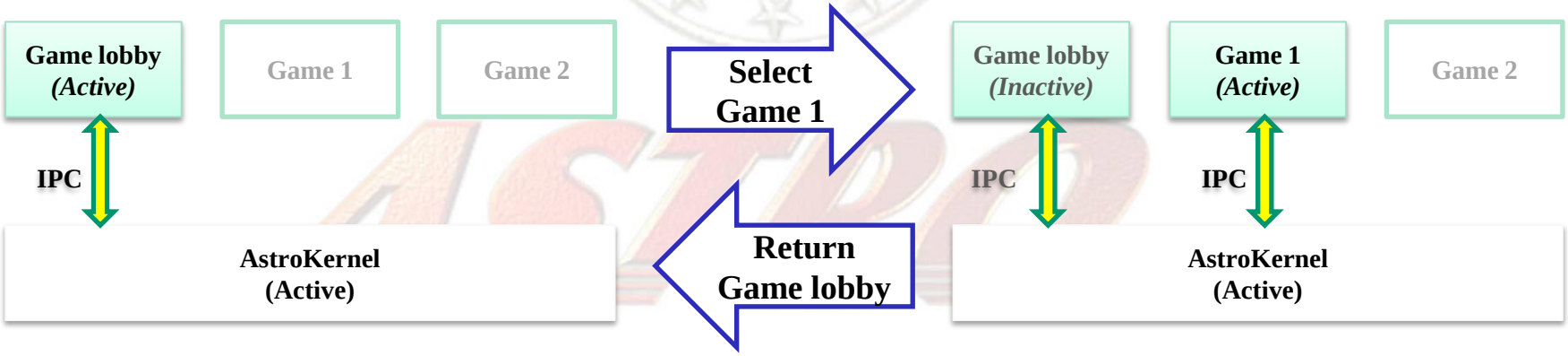
- AstroKernel version 2 (abbr. AK2)
  - Process “AK2” contains many modules to support contents (games) running, like process management, peripheral management, configurations/accounting/audit/gameplay services, non-volatile RAM access, network communication, etc.

VLT Software Architecture

- Content (a.k.a. downloadable content, game)
  - A content is a software program with its data, downloaded from central system.
  - Contents are classified into **Game**, **Attendant function**, **Game lobby**, **Attendant lobby**, and **Modules**.
  - Content is launched by AK2 as a child process. Content and AK2 communicate with each other through inter-process communication (IPC).

VLT Software Architecture

- Only Game lobby process with none or at most one content process is being run at the same time.



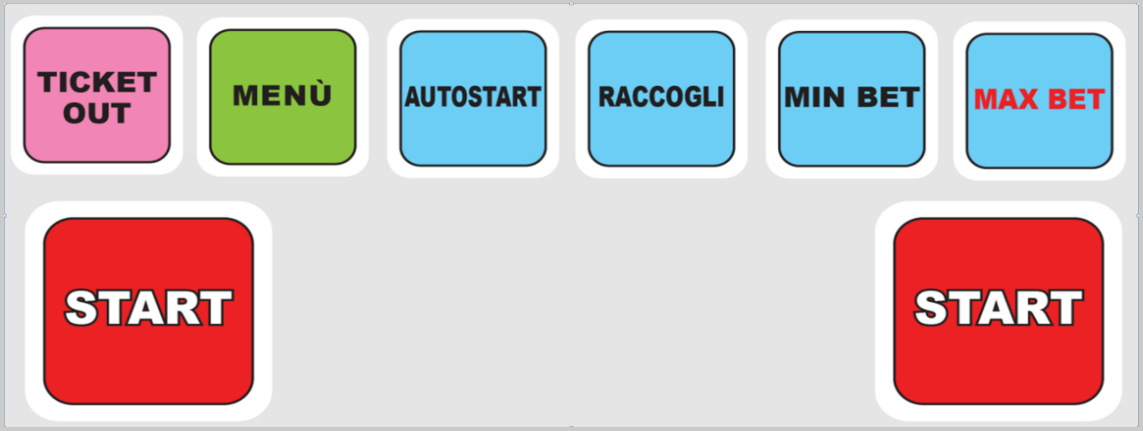
VLT Software Architecture

- Runtime environment

Item	Specification
Logic Board	PSM G920, GX-217, 2 cores 8GB RAM
OS	CentOS 7 64 bits with Kernel 4.18 (Embedded)
Monitors	Two monitors are arranged in upper and lower. Each is with resolution 1920x1080 pixels.
Touch Panel	One touch panel covers only the lower monitor.
Video	Graphics engine HD8000, X11, OpenGL 4.5.0
Audio	Onboard 5.1 channel
Payment device	Ticket printer, Bill acceptor

VLT Software Architecture

- Runtime environment (Cont.)

Item	Specification
Buttons and Backlights	
Storage	128GB SSD
NVRAM	8K bytes for each game

AstroGDK – Introduction

- Astro Game Development Kits (AstroGDK) provides a software environment including header files, libraries, samples, toolkits, simulator, and documents, for developer to develop and test games.

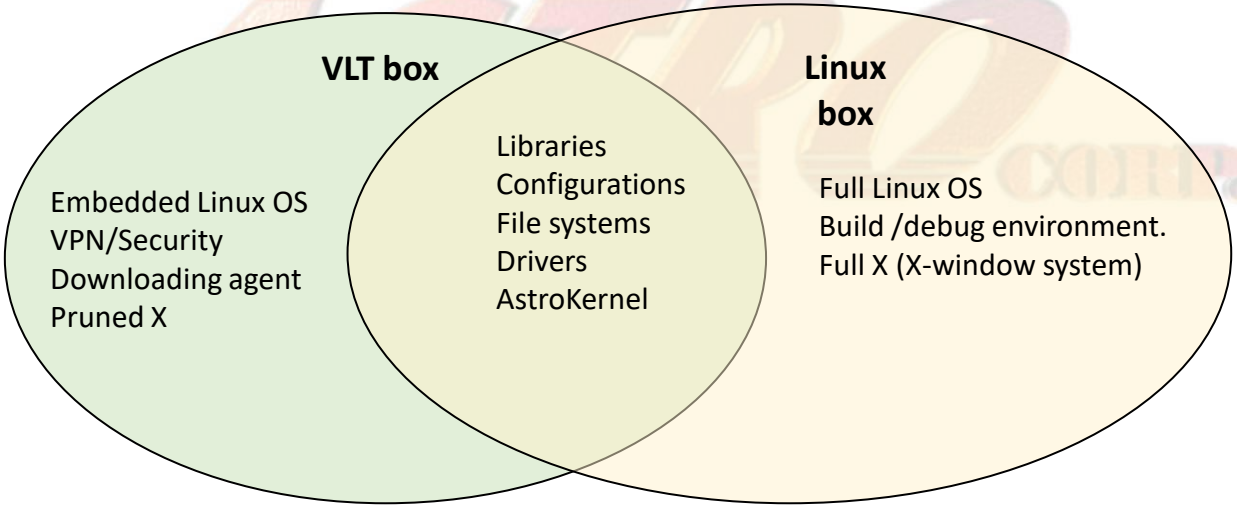
- Develop games run on platforms of Windows and Linux. Astro VLT runs Linux. So only need the Linux-version games eventually.

- Linux development machine (Linux box)
  - The same hardware with VLT box
  - Build, run, test your game

AstroGDK – Introduction

- Development models
  - Win box and Linux box (suggestion)
  - Linux box only
  - Linux VM

- Differences between Linux box and VLT box



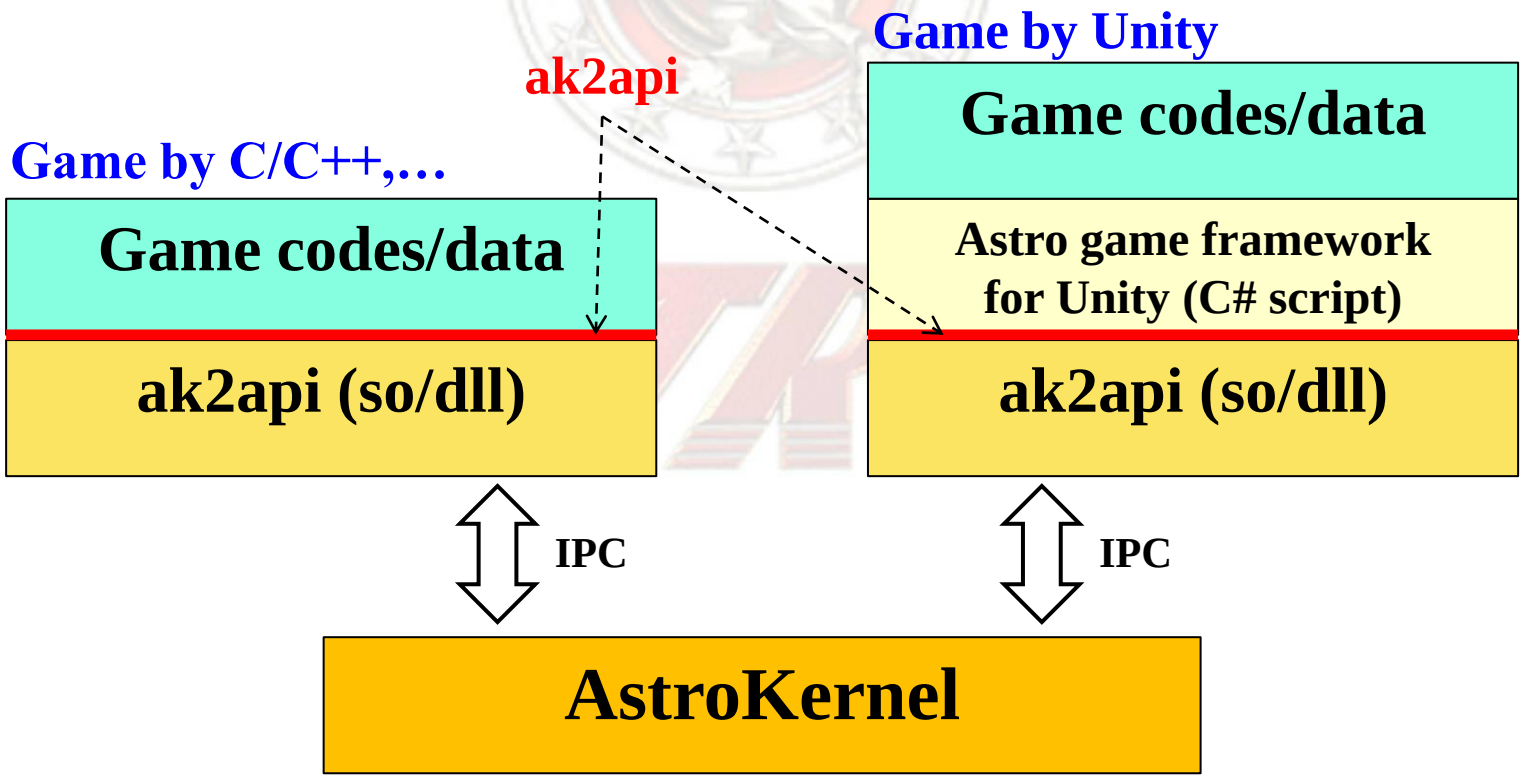
AstroGDK – Introduction

- API for game development:
  - Interface “ak2api”: a message-based software API composed of few C functions and a number of C structures.
  - It is wrapped in dll/so file format, so for those programming languages which can access C or dll/so file, like C / C++ / Python / C# / JS, etc., may apply the API to develop games.

- All communications between game program and the system are totally and only through the API.

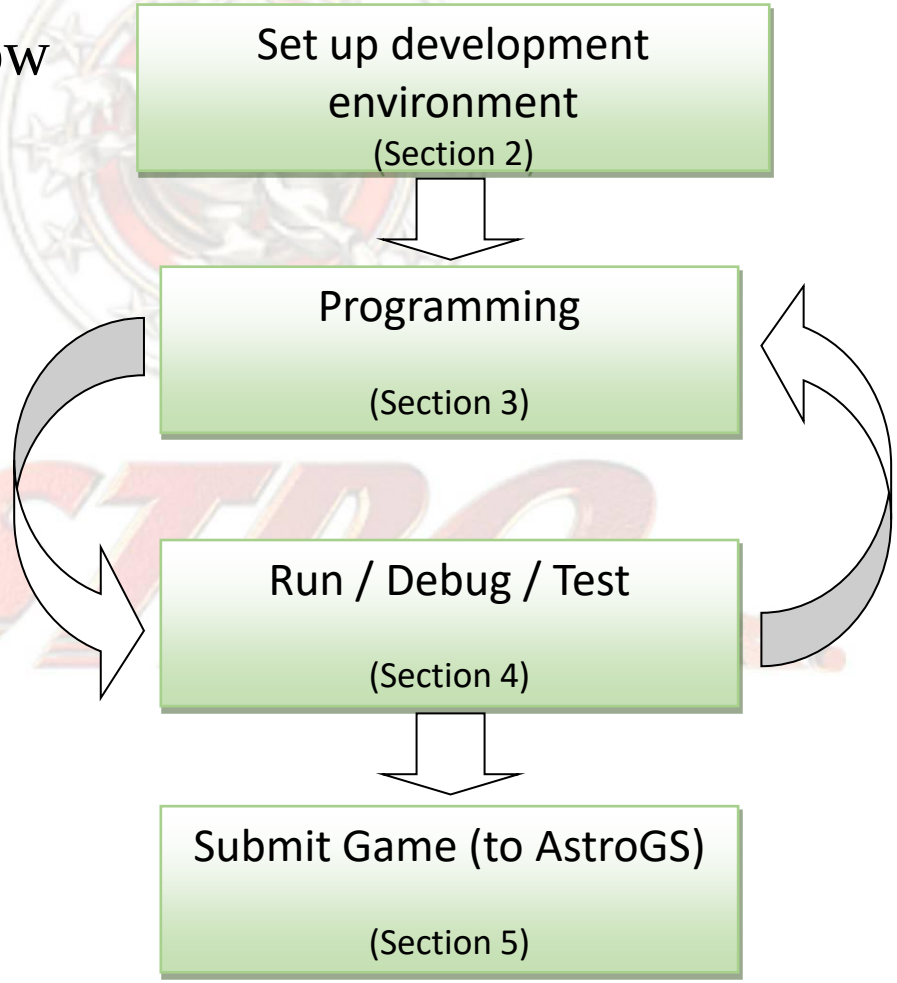
AstroGDK – Introduction

- AstroKernel exports client-end application programming interface “ak2api”. (C header file “ak2api.h”)



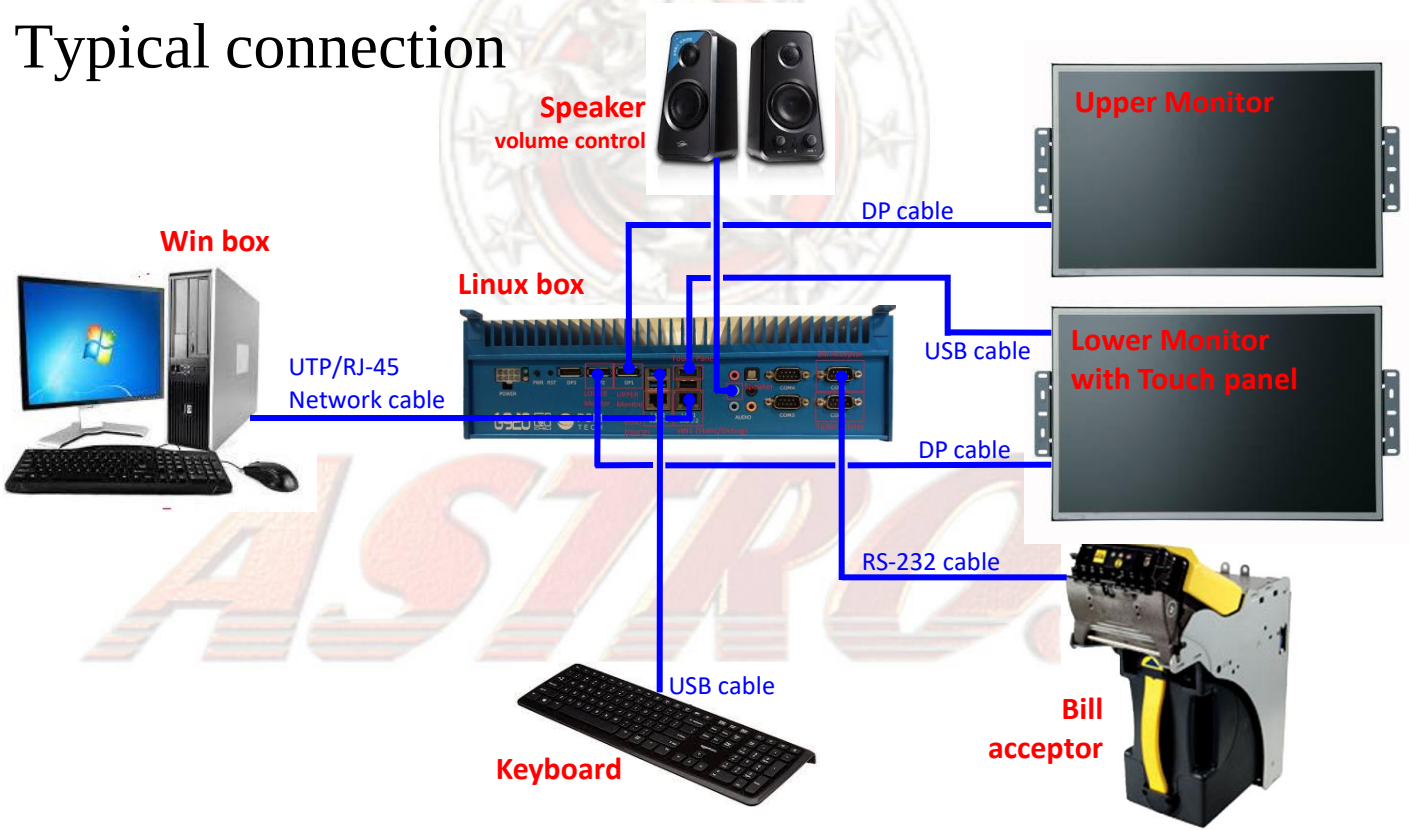
AstroGDK – Introduction

- Development flow and sections in Document.



AstroGDK – Setup (PSM G920)

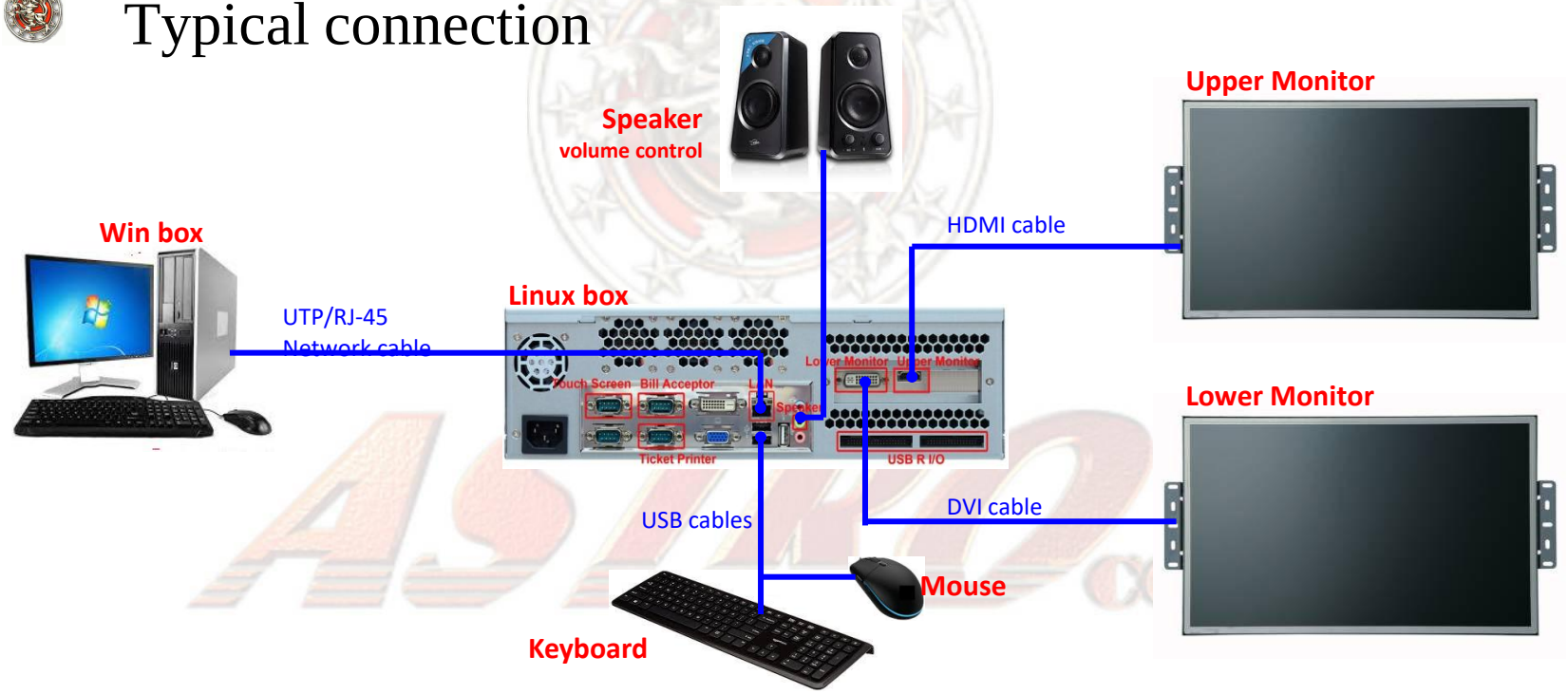
- Typical connection



- Configure the monitors and networks

AstroGDK – Setup (Astro H61)

- Typical connection



AstroGDK – Setup

- Steps to setup and program in developer's Windows machine (Win box)
  - Install and setup AstroGDK.
  - Set up directory of game source. Note, root directory of GDK and root directory of game source must be under the same directory (eg. MyProjects\) in parallel.
  - Start to program, run, test on Win box

Win box

Step 1: setup environment

MyProjects
- **AstroGDK**
- **my_game_project1**
- **my_game_project2**
- **…**

Step 2:
ssh to Linux box

Step 3: run mount_GameDev.sh to mount (bind) remote directory

Linux box

~/Projects/
- **GameDev/**
- **start.sh***
- **mount_GameDev.sh***

AstroGDK – Setup

- Steps to develop in Linux development machine (Linux box)
  - Config upper directory (eg. MyProjects\) as share folder (by network neighborhood/CIFS).
  - Use ssh client (eg. putty) to connect to Linux box.

Win box

Step 1: setup environment

MyProjects
- **AstroGDK**
- **my_game_project1**
- **my_game_project2**
- **…**

Step 2:
ssh to Linux box

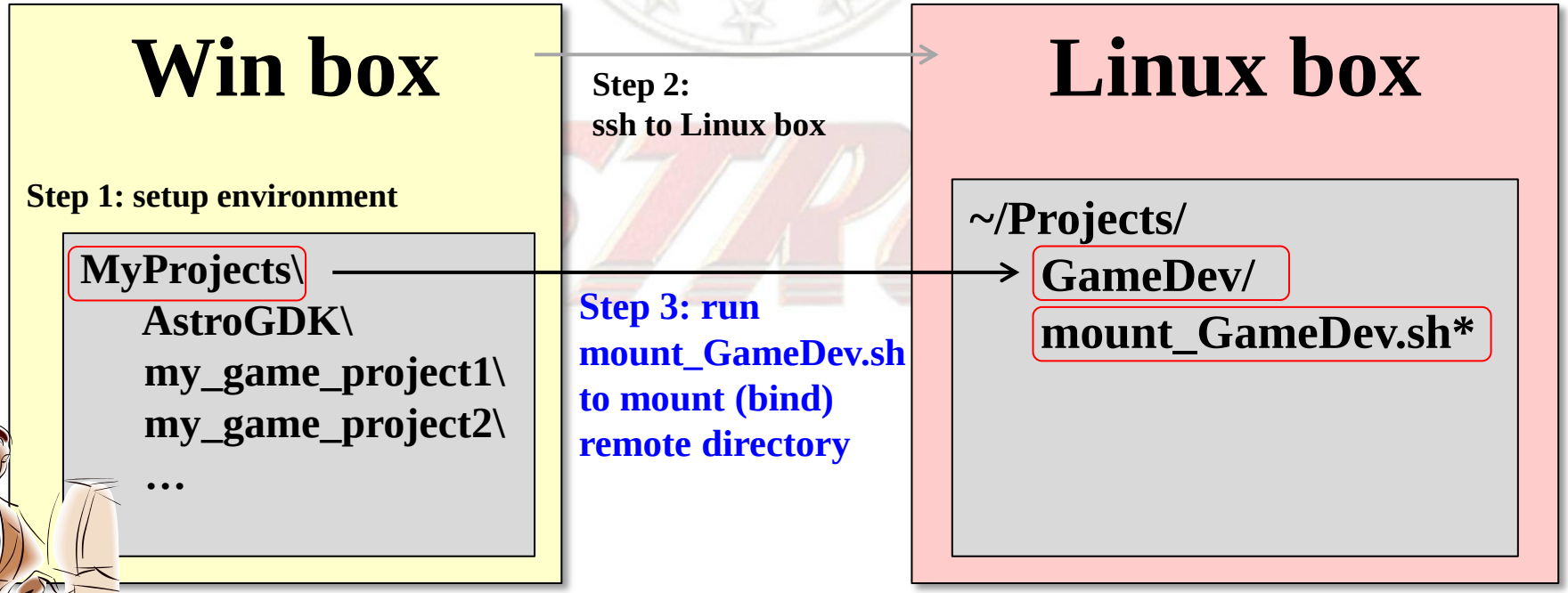
Step 3: run mount_GameDev.sh to mount (bind) remote directory

Linux box

~/Projects/
- **GameDev/**
- **mount_GameDev.sh***

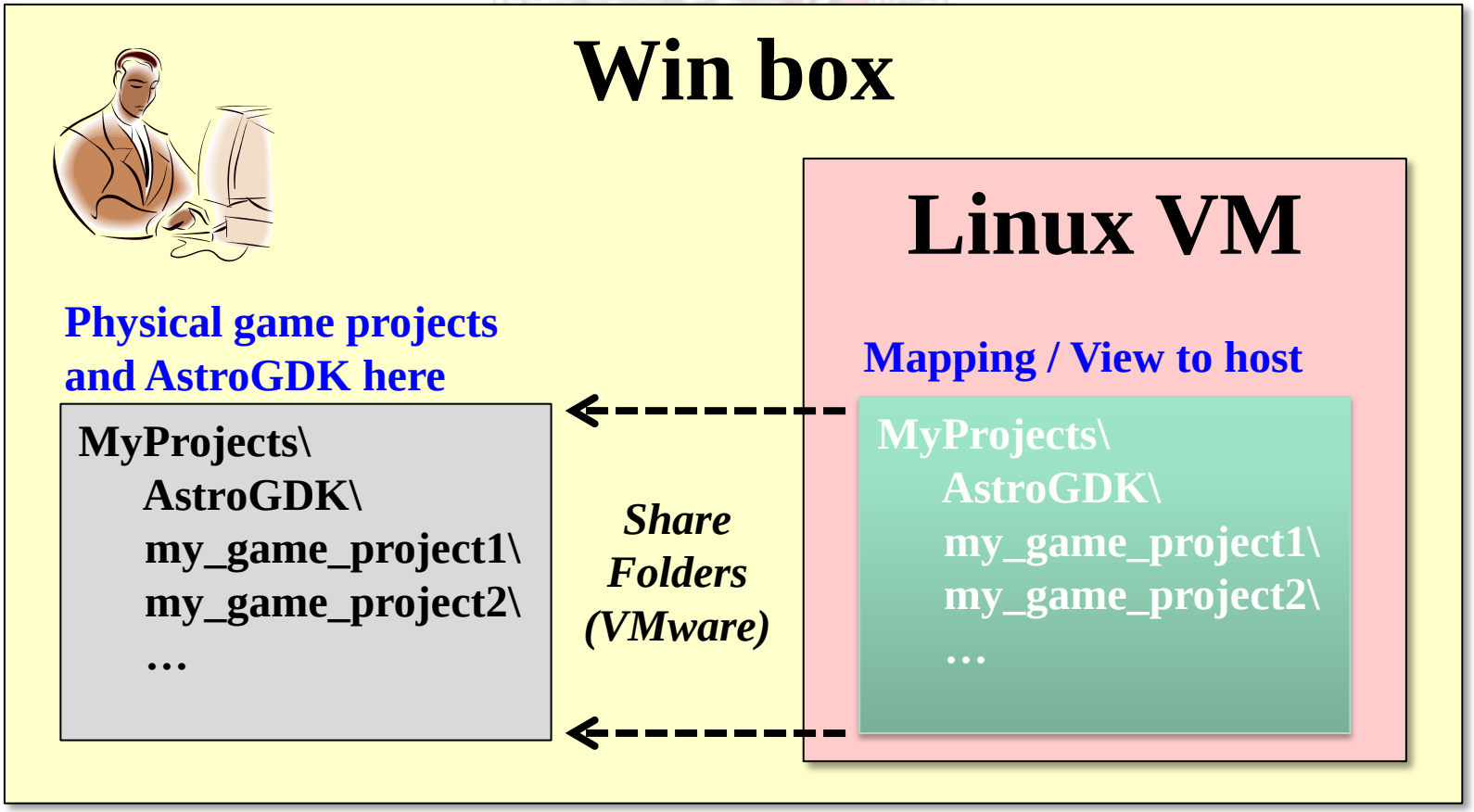
AstroGDK – Setup

- Steps to develop in Linux box (cont.)
  - Execute *mount_GameDev.sh* to mount remote shared folder in Win box (eg. *Work)* to here mount-point *~/Projects/GameDev/*.
  - Start to develop, run, test on Linux box
  - After game compiled, may execute *start.sh* to launch AK2 and then Game lobby.



AstroGDK – Setup (Linux VM)

- Steps to develop in Linux VM
  - Use ‘Shared Folder’ to access project folder from Linux VM.



AK2API – Programming

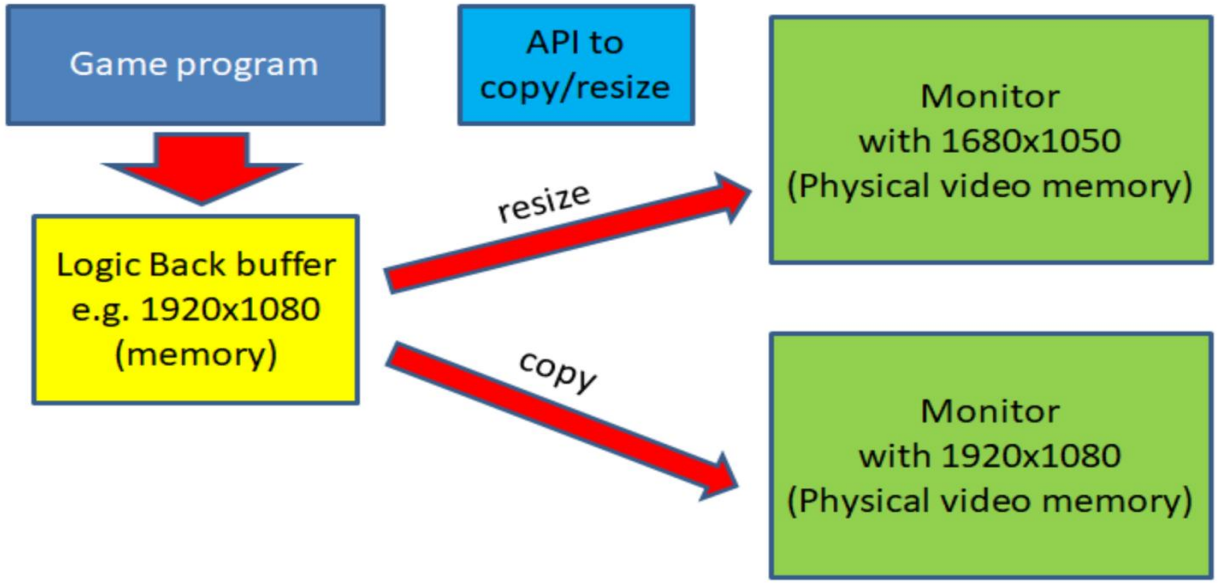
- Header files (C/C++ language)

```
#include "ak2api.h"              /* main header file of ak2api */
#include "ak2api_market_c6b.h"   /* market-dependent header file */
```

- Link your program with libak2api.so (or ak2api.dll)
- Terminate your program with defined exit codes.
- Run in a process, and non-exclusive window mode in full screen dimension.

AK2API – Programming

- Ability of running you game in different resolutions
  - 2D technology



  - 3D technology – apply the back buffer as texture

AK2API – Programming

- Functions
  - Initialize/release ak2api module and IPC:
    - *ak2api_init(…), ak2api_init_until_complete()*
    - *ak2api_cfg_get_item(…)*
    - *ak2api_exit()*
  - Transmit/receive messages:
    - *ak2api_get_message(void *p_msg)*
    - *ak2api_send_message(const struct ak2msg_base *p_msg)*
  - NVRAM handling:
    - `ak2api_nvbuf_get_buffer(char **pp_nvbuf_buffer,…)`
    - `ak2api_nvbuf_commit()`
    - `ak2api_nvbuf_if_synced()`

AK2API – Programming

- Messages
  - In form of C `struct`, containing size, message code, and following fields.

```
// Definition of base/head of a message structure.
// A complete message structure is this structure tailed optional part.
struct ak2msg_base
{
    int size;           // byte size of the complete message structure
    char code[27];      // message code/string here. (note! case sensitive)

    // optional/appened data from here
};
```

- Prefix of message code
  - “ac_”: the message is from AK2 to Content (game)
  - “ca_”: the message is from Content (game) to AK2

AK2API – Programming

- Process initialize/destroy (**init** / **exit**):
  - ak2api_**init**(…), ak2api_**init**_until_complete()
  - ak2api_**exit**()

- Configuration (**cfg**):
  - ak2api_**cfg**_get_item() to get configuration data.
  - "ac_**cfg**_updated"

- Process flow (**flow**):
  - "ca_**flow**_start", "ac_**flow**_terminate"
  - "ac_**flow**_suspend", "ac_**flow**_resume"

AK2API – Programming

- Game match (**game**):
  - “ca_**game**_start”, “ac_**game**_start_acked”
  - “ca_**game**_step”
  - “ca_**game**_end”, “ac_**game**_end_acked”
  - “ca_**game**_snapshot”, “ac_**game**_snapshot2”

- About odds (**rng** / **jp** / **outcome**):
  - “ca_**rng**_request”, “ac_**rng**_result”
  - “ca_**jp**_hit_check”, “ac_**jp**_hit_result”,
  - “ca_**jp**_award”, “ca_**jp**_award_acked”
  - “ca_**outcome**_request” , “ac_**outcome**_result”

AK2API – Programming

- Accounting (**credit** / **meter**):
  - “ac_**credit**_changed”
  - “ca_**credit**_payout”
  - “ca_**meter**_query”, “ac_**meter**_info”

- NVRAM buffer (**nvbuf**):
  - ak2api_**nvbuf**_get_buffer() to get pointer of NVRAM buffer.
  - ak2api_**nvbuf**_commit()
  - ak2api_**nvbuf**_if_synced()

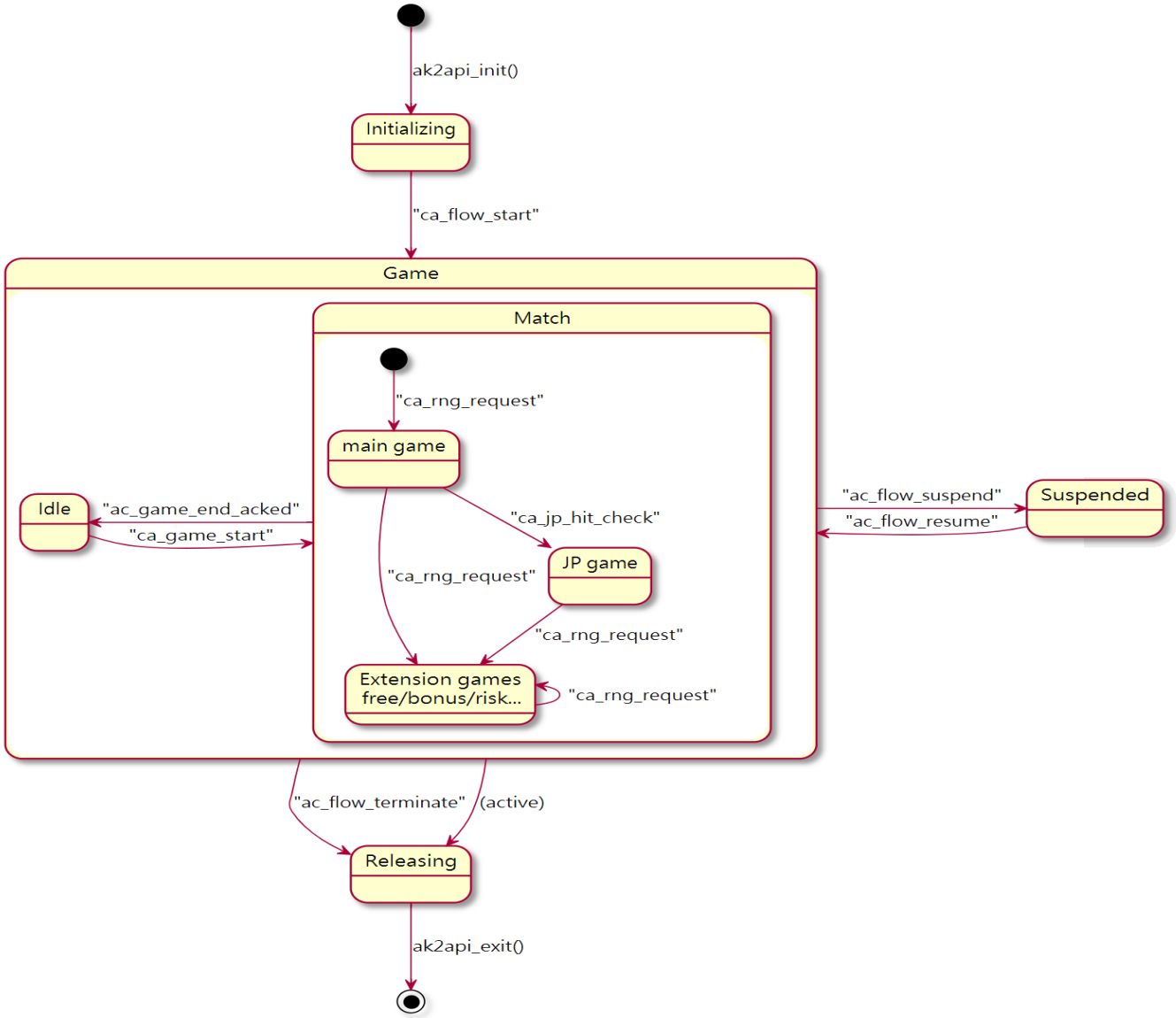
AK2API – Programming

- Low-level I/O (**touch** / **key** / **light**):
  - “ac_**touch**_pressed”, “ac_**touch**_released”
  - “ac_**key**_down”, “ac_**key**_up”
  - “ca_**light**_on”, “ca_**light**_off”

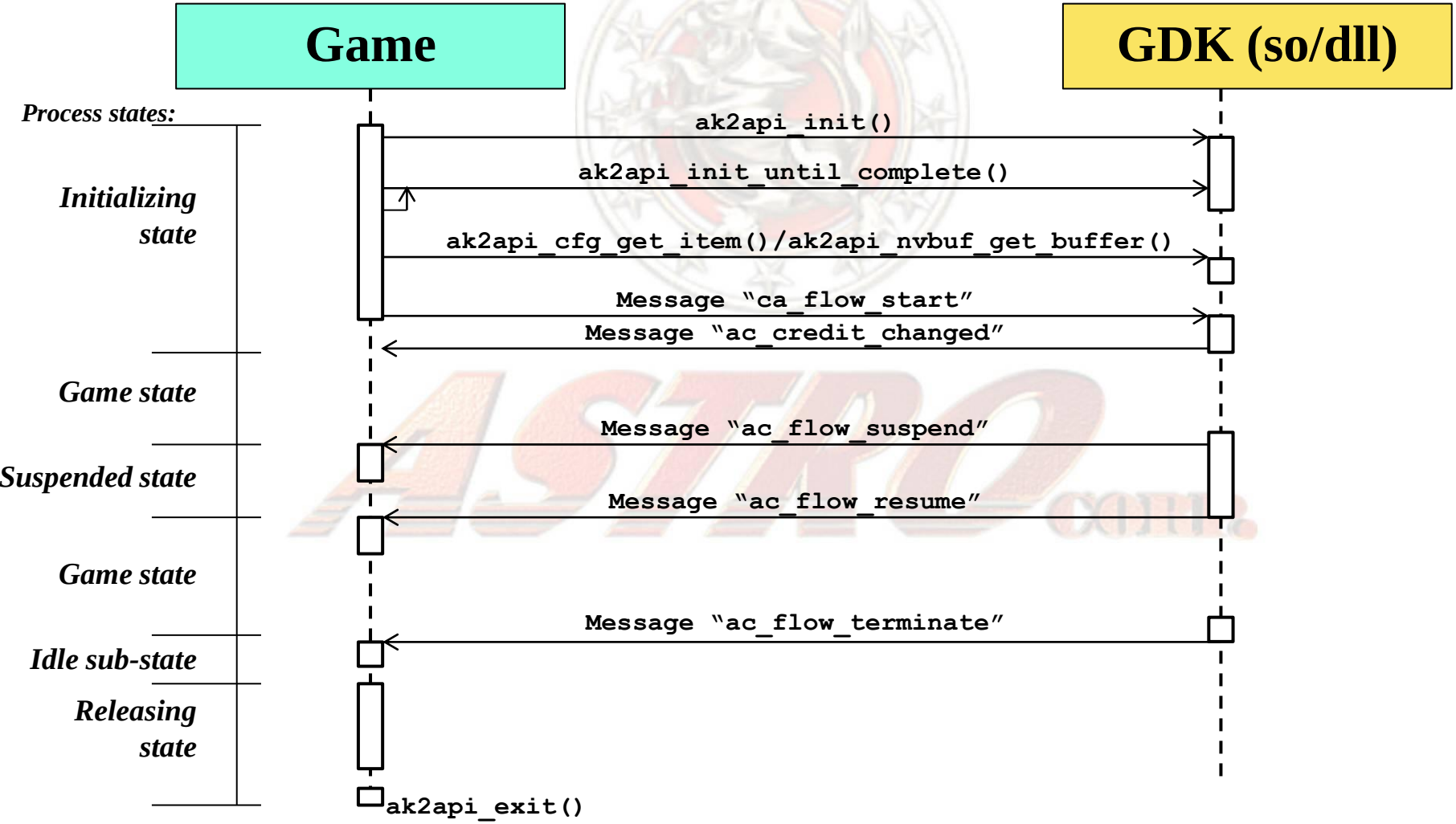
- Message exchange (**get** / **send**):
  - ak2api_**get**_message(), ak2api_**send**_message()

- (Others):
  - “ca_jp_broadcast”
  - “ca_atmosphere”, “ca_atmosphere2”

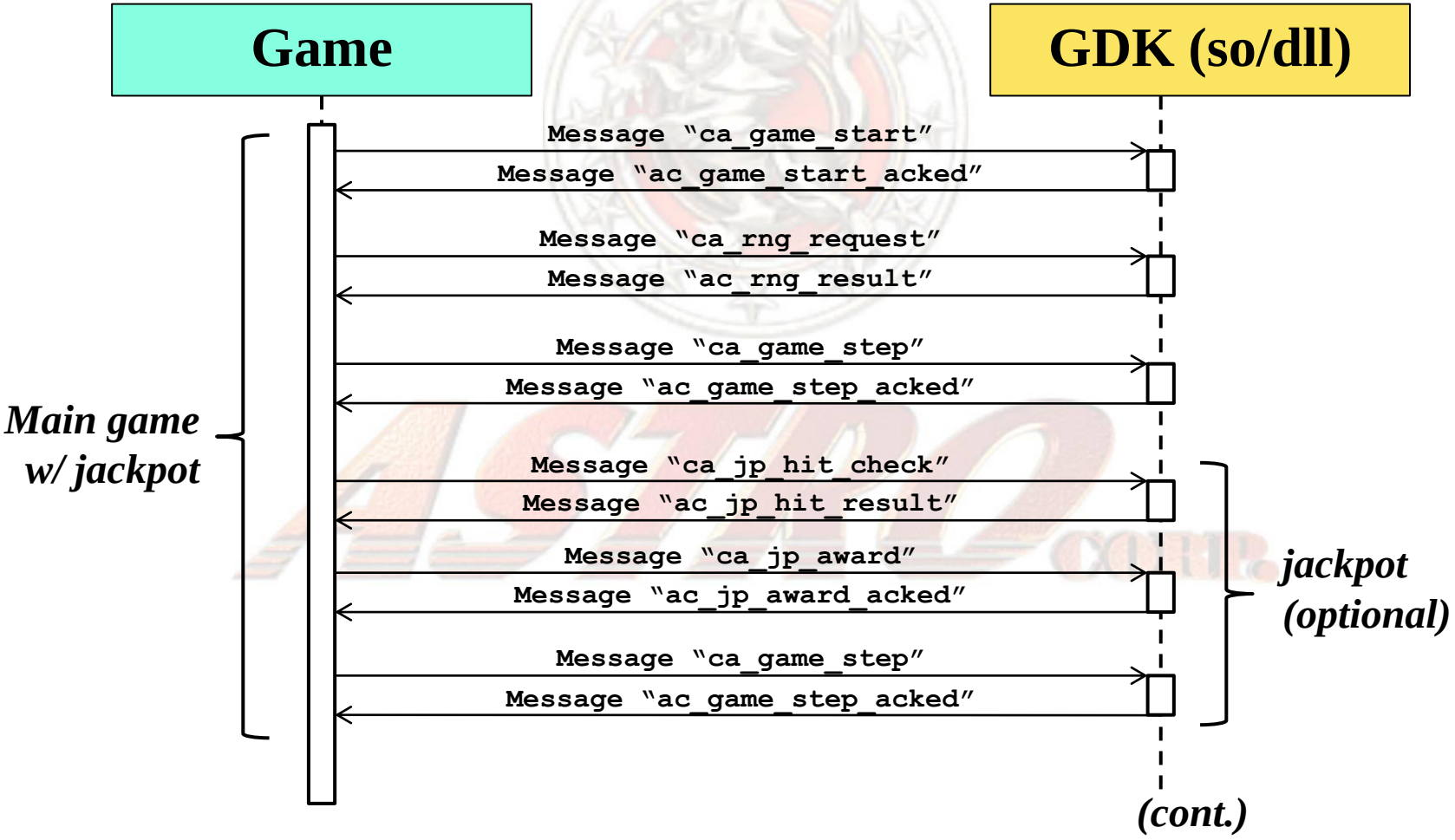
AK2API – State Machine



AK2API – main flow



AK2API – game flow (main/jp)

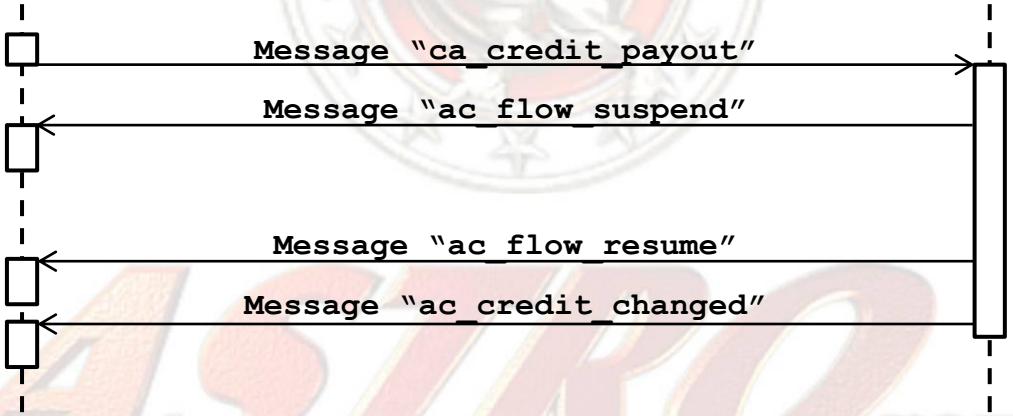


AK2API – game flow (step/end)

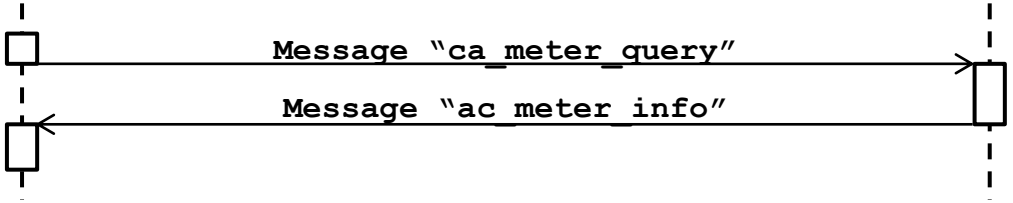
AK2API – accounting

Game | **GDK (so/dll)**

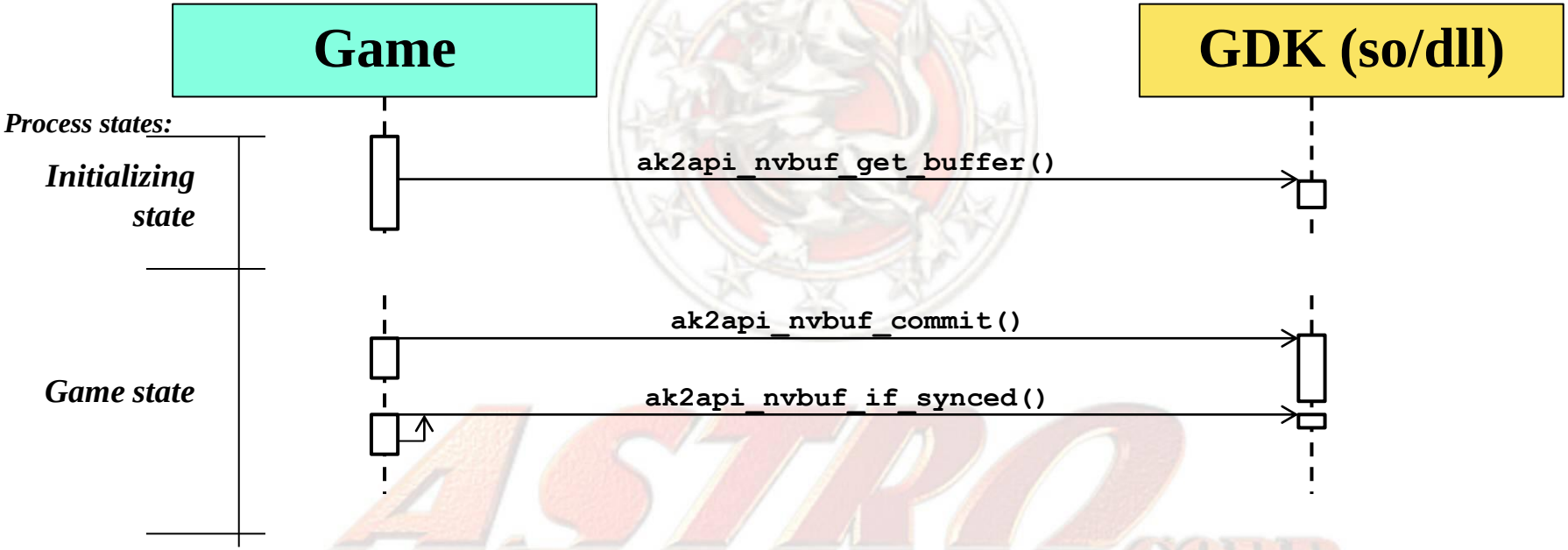
- Payout flow



- Query meters



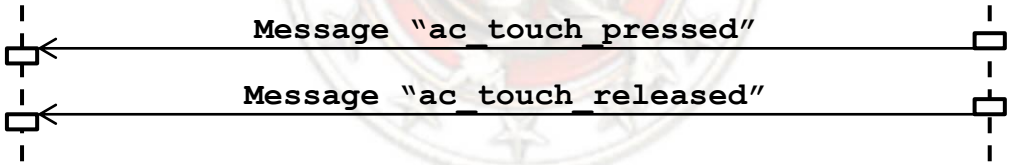
AK2API – nvram



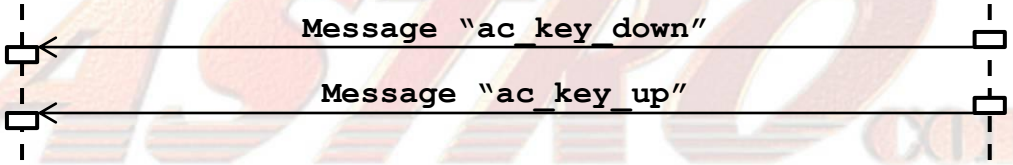
AK2API – low-level I/O

Game | **GDK (so/dll)**

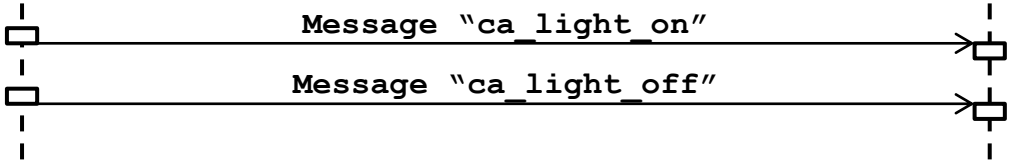
- Touch screen/panel



- Keys (physical buttons, switch keys)



- Lights (button / top /decorate lights)



AK2API – other issues

- Reporting game outcome in defined format
  - Game steps – report of main game, free game, bonus game, etc.
  - Game end – report of total bet and won.
- Incidents, Responsible gaming (Money/Time), Tax info
  - All controlled and displayed by AstroKernel
  - Game is suspended during message period.
- Use only necessary system resources, e.g. CPU and FPS
- Sound volume control
  - Sound material and hardware control

AK2API – Sample (init/destroy)

```
struct ak2msg_base msg;
int result_code;

...

// Initialize ak2api module
if ( ( result_code = ak2api_init() ) < 0 )
    error handling ...
while ( ( result_code = ak2api_until_complete() ) == 1 )
    wait a second or do other things.
if ( result_code < 0 )
    error handling ...

// other initialization here
...

// Tell AK2 that initialization of content has done.
memset( &msg, 0, sizeof *msg );
strcpy( msg.code, "ca_flow_start" );
msg.size = (int)sizeof *msg;
if ( ( result_code = ak2api_send_message( &msg ) ) < 0 )
    error handling ...

// Main loop here
...

// Release resource of content
...
ak2api_destroy();
exit( 0 );
```

AK2API – Sample (main loop)

```
struct ak2msg_base msg;
char msg_buf[AK2API_MAX_MESSAGE_SIZE];
int result_code;

...

// Main loop here
while ( ... )
{
    if ( ( result_code = ak2api_get_message( msg_buf ) ) < 0 )
        // error handling or exit main loop ...

    // if message received, ...
    if ( result_code > 0 )
    {
        // Refer to 'msg' here
        if ( 0 == strcmp( ((struct ak2msg_base *)msg_buf)->code, "ac_touch_pressed" ) )
            ...
        else if ( 0 == strcmp( ((struct ak2msg_base *)msg_buf)->code, "ac_touch_released" ) )
            ...
        ...
    }
    ...
}
```

AstroGDK – Run and Test

- Main execution binary must be named of “app” (Linux) or “app.exe” (Windows)
- Customize configuration files of AstroKernel
  - VLT.xml, list.xml, hwlayout.xml, debug.xml
- Launch your game from Game lobby
  - Add/modify item into list.xml
  - Start your game by click the icon in UI of game lobby
- Launch your game from command line (directly)

```
# start.sh <path_of_game>   [enter]
```

AstroGDK – Run and Test

- Simulate devices and events
  - Buttons – Keyboard
  - Touch panel – Mouse
  - Bill acceptor – keyboard to insert bill
  - Ticket printer – (none)
  - Incident – Keyboard to simulate door open/close
- Connection with real or simulation devices
  - Controlled by debug.xml
- Responsible gaming configurations
  - Configure through UI of Game lobby

AstroGDK – Run and Test

- Helper interface of feeding random numbers manually and monitoring the control states (lights control).
  - telnet, TCP/IP listen port 10000
- Log for debugging
  - While starting AstroKernel from batch script “start.sh” or “start.bat”, any messages output to **stdout** (standard output) and **stderr** (standard error) will be output to log file “AstroGDK\Runtime\data\ask.log”.
- Clean NVRAM files to initial the state

AstroGDK – Game Submission

- Game submission package. E.g. Game “Atlantis” version 1.3.0.4
- logo.png – for Downloader, icon.png, icon.webm, boundcing.webm, movie.avi – for Game lobby
- Above three media must follow defined name and format.
- Submit to test lab with **Game submission form** and **Game submission package**.

```
Atlantis_1.3.0.4\
  logo.png
  Distribution\
    app
    audio_data\
    video_data\
    scripts\
    ...
  media\
    icon.png
    icon.webm
    bouncing.webm
    movie.avi
```

AstroGDK – QA